THzSim2 Time-Domain Fitting and Oblique-Incidence Optical Response

THzSim2 Repository

April 19, 2026

1 Purpose and Scope

This note documents the calculations used by the current `thzsim2` code path for

1. building a physical sample stack from notebook-facing layer definitions,
2. computing the sample optical response at arbitrary angle of incidence for explicit `s` or `p` polarization,
3. propagating a measured or generated reference trace through that response, and
4. fitting unknown sample parameters directly in the time domain.

The present implementation is intentionally scoped to **isotropic scalar multilayers**. It does not model birefringence, anisotropy, Jones-matrix polarization conversion, or a full ellipsometric instrument train. The measured reference trace is treated as the source term of the forward model, so PCA emission, detector response, and any common-path optics outside the sample are absorbed into that measured reference.

2 Code Path Overview

The end-to-end notebook path is

`generate_reference_pulse / load_reference_csv` → `prepare_reference` → `build_sample` → `simulat`

The core implementation is distributed over these modules:

- `thzsim2/core/fft.py`: Fourier transform convention and inverse transform.
- `thzsim2/core/materials.py`: dielectric functions and passive $\tilde{n} = n + ik$ extraction.
- `thzsim2/core/transfer.py`: Fresnel coefficients, oblique propagation, single-layer Fabry–Perot sums, direct-interface handling, and arbitrary multilayer recursion.
- `thzsim2/core/forward.py`: measurement normalization and $E_{sam}(\omega)$ construction from the measured reference.
- `thzsim2/core/fitting.py`: parameter mapping, objective evaluation, optimization, and local covariance estimation.
- `thzsim2/workflows/sample_workflow.py`: notebook-facing sample resolution.
- `thzsim2/workflows/reference.py`: notebook-facing reference workflow.

3 Reference Trace and Fourier Convention

3.1 Time axis and trace

Let the uniformly sampled reference trace be

$$E_n = E(t_n), \qquad t_n = t_0 + n\Delta t, \qquad n = 0, \ldots, N-1.$$

The reference may come from `generate_reference_pulse(...)` or from a measured CSV imported by `load_reference_csv(...)` and normalized/exported by `prepare_reference(...)`.

3.2 Forward transform

`fft_t_to_w(trace, dt, t0)` in `thzsim2/core/fft.py` computes

$$\omega_k = 2\pi f_k,$$

on the standard FFT angular-frequency grid returned by `make_omega_grid`, and

$$\hat{E}(\omega_k) = \Delta t\, N\ \mathrm{IFFT}(E_n) \exp(i\omega_k t_0).$$

The explicit t_0 factor is important because measured traces are usually not centered at zero time.

3.3 Inverse transform

`ifft_w_to_t(spectrum, dt, t0)` applies

$$E_n = \frac{1}{N\Delta t}\,\mathrm{FFT}\left[\hat{E}(\omega_k)\exp(-i\omega_k t_0)\right].$$

This pair is the exact convention used by the forward model and the time-domain fitter.

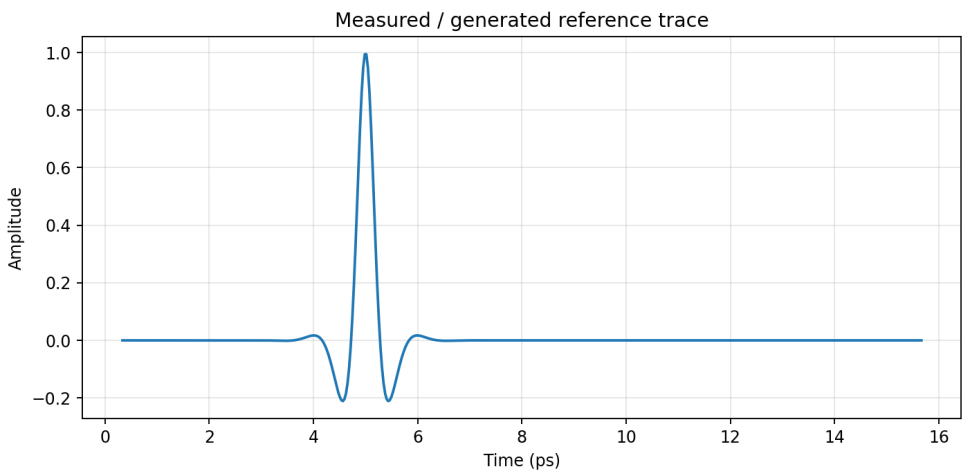

Figure 1: Reference trace generated by the documentation figure script. This is the waveform that enters the forward model before the sample response is applied.

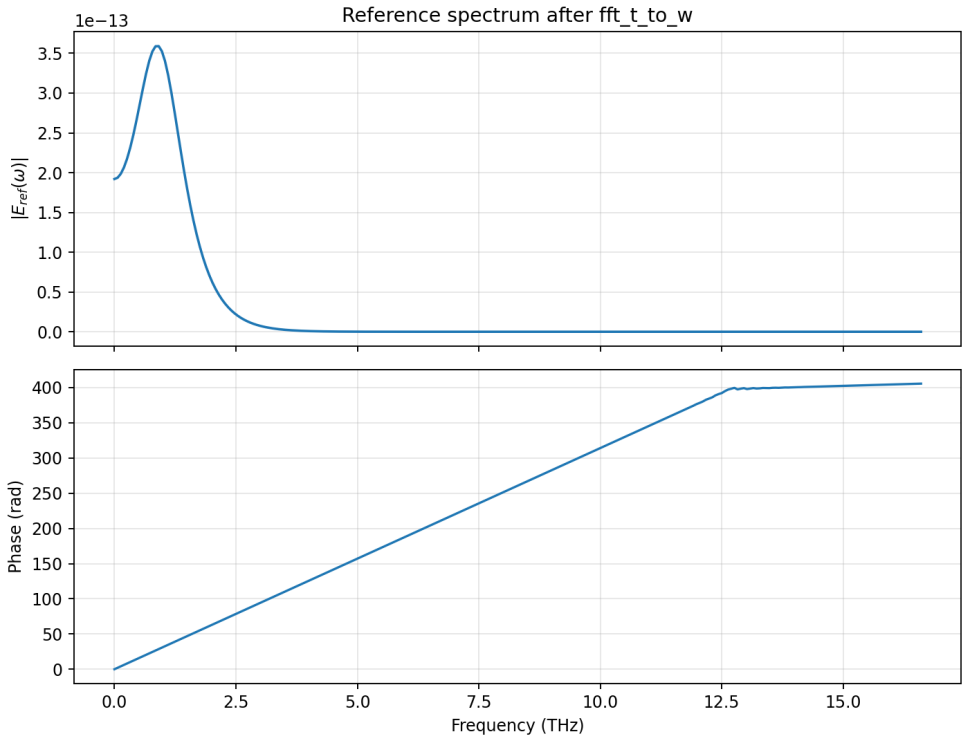

Figure 2: Magnitude and unwrapped phase of the reference spectrum after `fft_t_to_w`.

4 Sample Resolution and Material Models

4.1 Notebook-facing layer definitions

`build_sample(...)` accepts notebook objects such as `Layer`, `ConstantNK`, `Drude`, `Lorentz`, and `DrudeLorentz`. The helper `_resolve_value(...)` in `sample_workflow.py` converts each scalar or `Fit(...)` entry into

1. a concrete initial parameter value stored in the resolved stack, and
2. an entry in `sample.fit_parameters` when the quantity is a fit variable.

If a fit parameter is declared on a material field such as `layers[0].material.gamma_thz`, the fitter maps it into the resolved stack by the path conversion implemented in `stack_path_from_user_path(...)` and `apply_fit_values(...)`.

4.2 Dielectric functions

The material models in `thzsim2/core/materials.py` are

$$\varepsilon_{\text{Drude}}(\omega) = \varepsilon_\infty - \frac{\omega_p^2}{\omega(\omega + i\gamma)},$$

$$\varepsilon_{\text{Lorentz}}(\omega) = \varepsilon_\infty + \Delta\varepsilon \frac{\omega_0^2}{\omega_0^2 - \omega^2 - i\gamma\omega},$$

and

$$\varepsilon_{\text{Drude-Lorentz}}(\omega) = \varepsilon_\infty - \frac{\omega_p^2}{\omega(\omega + i\gamma)} + \sum_m \Delta\varepsilon_m \frac{\omega_{0,m}^2}{\omega_{0,m}^2 - \omega^2 - i\gamma_m\omega}.$$

The complex refractive index is obtained by

$$\tilde{n}(\omega) = n(\omega) + ik(\omega) = \sqrt{\varepsilon(\omega)},$$

with the passive branch selected by `nk_from_eps(...)` so that

$$k(\omega) \geq 0.$$

4.3 Imported refractive-index files

For `NKFile`, `build_sample(...)` loads a `freq_thz,n,k` CSV and uses `_linear_interp_extrap(...)` and `_resolve_imported_nk(...)` to align the tabulated n, k values to the working positive-frequency grid.

5 Oblique-Incidence Optics

5.1 Measurement configuration

The measurement state is represented by the notebook-facing dataclasses `Measurement` and `ReferenceStandard` in `thzsim2/models/measurement.py`. The public fields are

- `mode`: ̈`transmission`̈ or ̈`reflection`̈,
- `angle_deg`: incidence angle in the incident medium,
- `polarization`: ̈`s`̈ or ̈`p`̈,
- `reference_standard`: either `identity` or an explicit stack.

5.2 Tangential-wavevector conservation and branch choice

For an isotropic stack, the tangential component of the wavevector is conserved. If the incident medium is $\tilde{n}_0$ and the incidence angle is θ_0,

$$\tilde{n}_j \sin\theta_j = \tilde{n}_0 \sin\theta_0.$$

The code works with

$$\tilde{n}_{\parallel} = \tilde{n}_0 \sin\theta_0, \qquad \cos\theta_j = \sqrt{1 - \left(\frac{\tilde{n}_{\parallel}}{\tilde{n}_j}\right)^2}.$$

Because $\tilde{n}_j$ can be complex, the square root has two branches. The helper `_forward_cos_theta(...)` chooses the branch for which the normal propagation factor

$$\tilde{n}_j \cos\theta_j$$

corresponds to a forward-decaying or forward-propagating wave:

$$\Im(\tilde{n}_j \cos\theta_j) \geq 0,$$

and if the imaginary part is numerically zero it also requires

$$\Re(\tilde{n}_j \cos\theta_j) \geq 0.$$

5.3 Oblique Fresnel coefficients

For the interface from medium j to medium k, `fresnel_t_oblique(...)` and `fresnel_r_oblique(...)` use

$$t_{jk}^{(s)} = \frac{2\tilde{n}_j \cos\theta_j}{\tilde{n}_j \cos\theta_j + \tilde{n}_k \cos\theta_k}, \qquad r_{jk}^{(s)} = \frac{\tilde{n}_j \cos\theta_j - \tilde{n}_k \cos\theta_k}{\tilde{n}_j \cos\theta_j + \tilde{n}_k \cos\theta_k},$$

$$t_{jk}^{(p)} = \frac{2\tilde{n}_j \cos\theta_j}{\tilde{n}_k \cos\theta_j + \tilde{n}_j \cos\theta_k}, \qquad r_{jk,\text{local}}^{(p)} = \frac{\tilde{n}_k \cos\theta_j - \tilde{n}_j \cos\theta_k}{\tilde{n}_k \cos\theta_j + \tilde{n}_j \cos\theta_k}.$$

Reported `p`-reflection convention. The implementation internally uses the standard local-basis Fresnel form above, but the final reported reflected `p` response is mapped into a fixed scalar measurement channel by `_physical_response(...)`:

$$H_{\text{reflection, reported}}^{(p)} = -H_{\text{reflection, local}}^{(p)}.$$

This convention makes the isotropic `s`/`p` responses collapse back together at normal incidence in the same scalar channel that a linearly polarized PCA-style measurement would use.

5.4 Propagation through one layer

For layer j with thickness d_j, the oblique propagation factor is

$$p_j(\omega) = \exp\left(i\omega\tilde{n}_j \cos\theta_j \frac{d_j}{c_0}\right),$$

as implemented by `_propagation_factor_oblique(...)`.

5.5 Direct interface and zero-thickness limit

If a stack contains no finite-thickness layers, the code now treats it as the direct interface between the incident and exit media:

$$H_{\text{transmission}} = t_{0,\text{out}}, \qquad H_{\text{reflection}} = r_{0,\text{out}}.$$

When $n_{\text{in}} = n_{\text{out}}$, this reduces to identity transmission and zero reflection, which is exactly the old empty-stack case.

5.6 Single-layer finite Fabry–Perot sum

For one physical layer, the code preserves an explicit finite-round-trip expansion when `max_internal_reflectio` is an integer:

$$z(\omega) = r_{10} p_1 r_{12} p_1,$$

$$F_M(\omega) = \sum_{m=0}^{M} z(\omega)^m.$$

Then

$$T(\omega) = t_{01} p_1 t_{12} F_M(\omega),$$

$$R(\omega) = r_{01} + t_{01} p_1 r_{12} p_1 t_{10} F_{M-1}(\omega).$$

This path is implemented in `_medium_stack_response_positive(...)`.

5.7 Arbitrary multilayer coherent recursion

For arbitrary multilayers the current exact coherent implementation uses the recursive scattering formulation in `_multilayer_response_recursive(...)` rather than the old normal-incidence matrix shortcut.

Let the physical layers be indexed by $j = 1, \ldots, N$, with incident medium 0 and exit medium $N + 1$. Define the effective downstream response seen from layer $j + 1$ as

$$R_{j+1}^{\text{eff}}(\omega), \qquad T_{j+1}^{\text{eff}}(\omega).$$

The recursion starts at the last interface:

$$R_N^{\text{eff}} = r_{N,N+1}, \qquad T_N^{\text{eff}} = t_{N,N+1}.$$

Then for $j = N - 1, \ldots, 0$,

$$T_j^{\text{eff}} = \frac{t_{j,j+1} p_{j+1} T_{j+1}^{\text{eff}}}{1 - r_{j+1,j} p_{j+1} R_{j+1}^{\text{eff}} p_{j+1}},$$

$$R_j^{\text{eff}} = r_{j,j+1} + \frac{t_{j,j+1} p_{j+1} R_{j+1}^{\text{eff}} p_{j+1} t_{j+1,j}}{1 - r_{j+1,j} p_{j+1} R_{j+1}^{\text{eff}} p_{j+1}}.$$

The overall stack response is

$$H_{\text{transmission}} = T_0^{\text{eff}}, \qquad H_{\text{reflection}} = R_0^{\text{eff}},$$

followed by the reported-`p` reflection sign convention above.

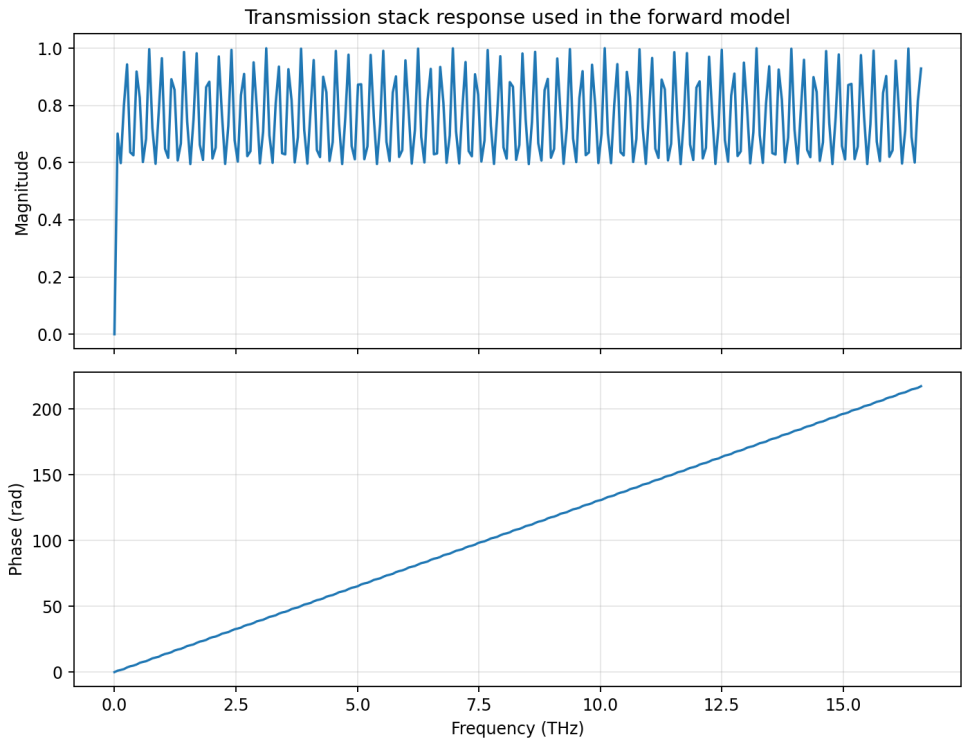

Figure 3: Transmission worked example: the complex transfer function used in `simulate_sample_from_reference`.

6 Forward Model with Measured Reference Normalization

6.1 Reference-standard normalization

The central forward-model equation implemented in `thzsim2/core/forward.py` is

$$E_{\text{sam}}(\omega) = E_{\text{ref,meas}}(\omega)\frac{H_{\text{sample}}(\omega)}{H_{\text{ref,std}}(\omega)}.$$

This is exactly the physical assumption used in the user plan:

- the measured reference trace contains the PCA source spectrum, detector response, and common-path optics,
- the modeled sample-plane response is factored in through $H_{\text{sample}}/H_{\text{ref,std}}$,
- the reference standard may be the identity or an explicit physical stack.

6.2 Implementation details

`simulate_sample_from_reference(...)` performs

1. `fft_t_to_w` on the reference trace,
2. `stack_response_function(...)` on the sample stack,
3. the same stack-response calculation on the reference-standard stack if one is supplied,
4. the ratio $H_{\text{sample}}/H_{\text{ref,std}}$,
5. multiplication by the measured reference spectrum,
6. `ifft_w_to_t` back to the time domain.

Reflection measurements have no silent default reference standard. This is enforced by `normalize_measurement(...)`. The helper `_validate_reference_standard_response(...)` also rejects normalization cases where $H_{\text{ref,std}}(\omega)$ becomes numerically too small.

7 Time-Domain Fitting

7.1 Parameter mapping

The fitter works on a flat vector

$$\mathbf{x} = (x_1, \ldots, x_p),$$

but the simulator requires a nested resolved-stack dictionary. The helper functions `_parse_path`, `_get_by_path`, `_set_by_path`, `stack_path_from_user_path`, and `apply_fit_values` perform the algebraic mapping

$$\mathbf{x} \longrightarrow \text{resolved stack}.$$

7.2 Synthetic observations and noise

For study runs and documentation examples, the code adds white Gaussian noise through `add_white_gaussian_noise(...)`. The standard deviation is derived from the requested dynamic range by

$$\sigma_{\text{noise}} = \frac{\max_t |E_{\text{sam,true}}(t)|}{10^{D/20}},$$

where D is the dynamic range in dB. This is implemented by `noise_sigma_from_dynamic_range(...)`.

7.3 Objective function

The primary time-domain objectives are

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}\left(E_{\mathrm{model}}(t_n) - E_{\mathrm{obs}}(t_n)\right)^2,$$

and

$$\text{relative } L^2 = \frac{\|E_{\mathrm{model}} - E_{\mathrm{obs}}\|_2}{\max(\|E_{\mathrm{obs}}\|_2, 10^{-30})}.$$

The selector `objective_metric_value(...)` chooses between them. The diagnostic `snr_db(...)` uses

$$\mathrm{SNR}_{\mathrm{dB}} = 20\log_{10}\left(\frac{\mathrm{RMS}(E_{\mathrm{signal}})}{\mathrm{RMS}(E_{\mathrm{noise}})}\right).$$

7.4 Optimization sequence

`fit_sample_trace(...)` uses a two-stage search:

1. bounded differential evolution,
2. bounded local refinement with `L-BFGS-B`.

Each trial vector $\mathbf{x}$ is mapped into a stack, simulated by `simulate_sample_from_reference(...)` with the current `Measurement`, and scored in the time domain.

7.5 Residual vector and covariance estimate

After the optimum $\hat{\mathbf{x}}$ is found, the code builds the residual vector

$$\mathbf{r}(\mathbf{x}) = E_{\mathrm{obs}} - E_{\mathrm{model}}(\mathbf{x}).$$

`_residual_vector(...)` concatenates real and imaginary parts only if the residual is complex; for the usual real time-domain fit, it remains a real vector of length N.

The finite-difference Jacobian used by `_estimate_parameter_sigmas(...)` is

$$J_{:,j} \approx \frac{\mathbf{r}(\hat{\mathbf{x}} + \Delta x_j \mathbf{e}_j) - \mathbf{r}(\hat{\mathbf{x}} - \Delta x_j \mathbf{e}_j)}{(\hat{x}_j + \Delta x_j) - (\hat{x}_j - \Delta x_j)}.$$

The step length is chosen by `_step_from_bounds(...)` from the parameter magnitude, the fit span, and the configured relative finite-difference step.

With m residual samples and p fitted parameters,

$$\hat{\sigma}^2 = \frac{\mathbf{r}^T\mathbf{r}}{\max(m-p, 1)}, \qquad \Sigma \approx \hat{\sigma}^2 (J^T J)^+,$$

where $(\cdot)^+$ is the pseudoinverse. The reported standard deviations and correlations are

$$\sigma_j = \sqrt{\Sigma_{jj}}, \qquad \rho_{ij} = \frac{\Sigma_{ij}}{\sigma_i \sigma_j}.$$

8 Edge Cases and Constraints

- **Isotropic only.** The code assumes scalar $\tilde{n}(\omega)$ in each layer.
- **One measured channel per run.** Each fit uses one geometry and one polarization channel.
- **Reflection requires an explicit standard.** This is enforced by `normalize_measurement(...)`.
- **Empty stack is a direct interface.** If $n_{\text{in}} \neq n_{\text{out}}$, a layer-free stack still has nontrivial Fresnel response.
- **Zero-thickness limit collapses to the direct interface.** This is handled in `single_layer_transfer(..`
- **Normal incidence collapses `s`/`p`.** The reported reflected `p` convention is chosen so the scalar channel is degenerate at $\theta_0 = 0$.
- **Single-layer finite reflection count.** The explicit integer `max_internal_reflections` truncation is exact for one layer. General multilayers use the exact coherent recursion.
- **Passive branch selection.** `nk_from_eps(...)` enforces $k \geq 0$.

9 Worked Example A: Transmission Time-Domain Fit

The documentation figure script `docs/generate_time_domain_fit_figures.py` builds a Drude film with these true values:

$$d = 182\,\mu\text{m}, \qquad \tau = 5.2\,\text{ps}, \qquad \sigma = 2.8 \times 10^{-2}\,\text{S}\,\text{m}^{-1},$$

at

$$\theta_0 = 30^\circ, \qquad \texttt{polarization} = \texttt{p}, \qquad \texttt{mode} = \texttt{transmission}.$$

The notebook-facing fit variables are thickness, Drude plasma frequency, and Drude damping frequency. The clean simulated sample trace is generated by

$$E_{\text{sam,true}}(\omega) = E_{\text{ref}}(\omega) H_{\text{sample}}(\omega),$$

Gaussian noise is added at 85 dB dynamic range, and the fit then minimizes the time-domain MSE.

Algebraic structure of the fit. For each optimizer call:

1. `apply_fit_values(...)` writes $\mathbf{x}$ into the resolved stack.
2. `stack_response_function(...)` computes $H_{\text{sample}}(\omega)$.
3. `simulate_sample_from_reference(...)` produces $E_{\text{model}}(t)$.
4. `objective_metric_value(...)` compares $E_{\text{model}}(t)$ to $E_{\text{obs}}(t)$.

This is repeated inside differential evolution and then inside the local bounded minimizer.

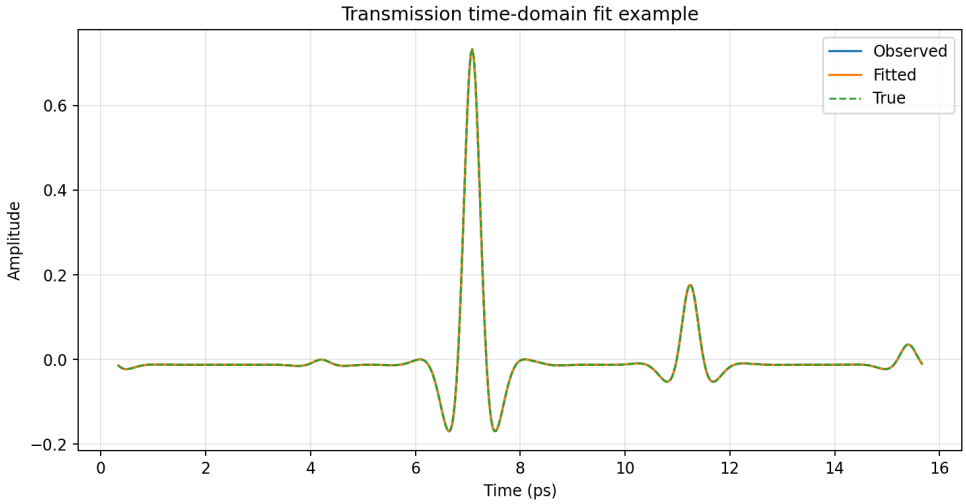

Figure 4: Transmission worked example: noisy observed trace, true trace, and recovered fitted trace.

Figure 5: Residual trace from the transmission worked example.

10 Worked Example B: Reflection with Explicit Reference Standard

The reflection example in the figure script uses

$$\theta_0 = 45^\circ, \qquad \texttt{polarization} = \texttt{s}, \qquad \texttt{mode} = \texttt{reflection},$$

for a thin coating above an exit half-space with

$$n_{\text{out}} = 1.52.$$

The reference standard is the bare interface itself, represented as an explicit stack with no physical layers and `n_out = 1.52`. The forward model therefore uses

$$E_{\text{sam}}(\omega) = E_{\text{ref,meas}}(\omega)\frac{H_{\text{coated interface}}(\omega)}{H_{\text{bare interface}}(\omega)}.$$

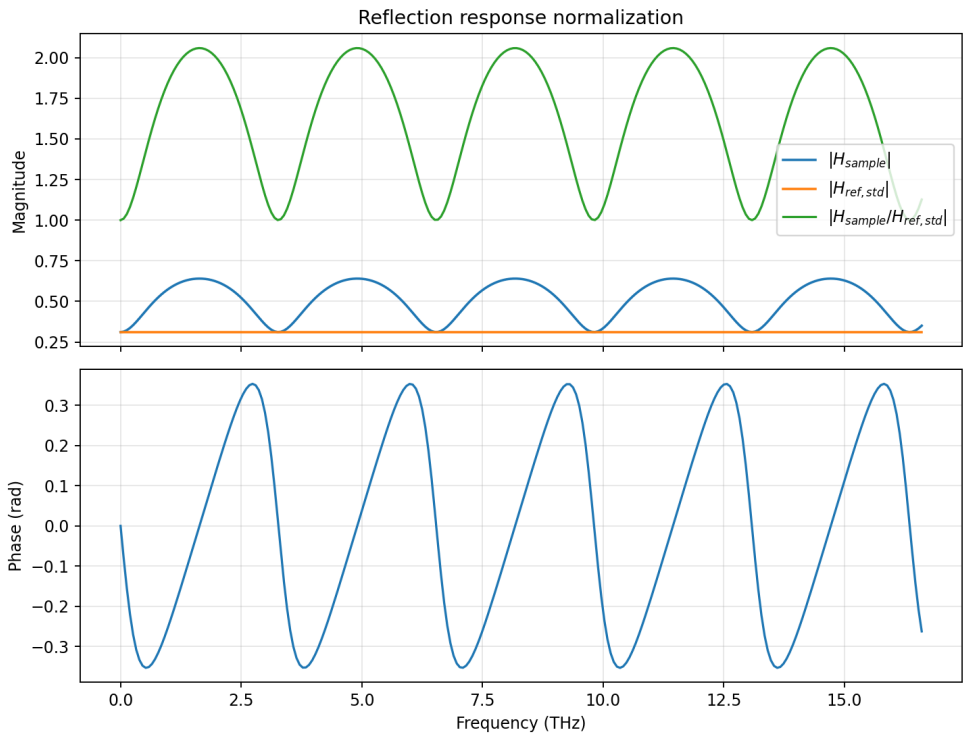

Figure 6: Reflection worked example: sample response, reference-standard response, and their ratio.

11 Code Crosswalk

Calculation block	Primary functions	Role in the pipeline
Reference generation / import	`generate_reference_pulse`, `load_reference_csv`, `prepare_reference`	Construct the measured or synthetic reference trace and its exported summary.

Calculation block	Primary functions	Role in the pipeline
Time and frequency grids	`make_time_grid`, `make_omega_grid`, `fft_t_to_w`, `ifft_w_to_t`	Define the discrete Fourier convention used by the forward model and inverse synthesis.
Pulse models	`gaussian_carrier_pulse`, `sech_carrier_pulse`, `make_pulse`	Generate the analytic source waveform when the reference is synthetic.
Material permittivity models	`eps_drude`, `eps_lorentz`, `eps_drude_lorentz`	Evaluate $\varepsilon(\omega)$ for supported isotropic material models.
Passive index extraction	`nk_from_eps`, `evaluate_material_nk`, `_nk_from_material_dict`	Convert dielectric functions into $n(\omega)$ and $k(\omega)$ on the working grid.
Sample resolution	`build_sample`, `_resolve_material`, `_resolve_value`	Turn notebook-facing sample definitions into a resolved simulation stack and fit registry.
Oblique-interface optics	`fresnel_t_oblique`, `fresnel_r_oblique`, `_forward_cos_theta`	Compute angle-aware interface amplitudes with a stable branch choice in complex media.
Layer propagation	`propagation_factor`, `_propagation_factor_oblique`	Apply the phase / attenuation factor across each layer.
Finite single-layer Fabry–Perot series	`_finite_fp_factor`, `_medium_stack_response_positive`	Evaluate explicitly truncated positive-trip sums when a single layer requests a finite count.
Exact multilayer response	`_multilayer_response_recursive`, `stack_response_function`	Compute, the exact coherent transmission or reflection of arbitrary isotropic multilayers.
Measurement normalization	`Measurement`, `ReferenceStandard`, `normalize_measurement`, `_reference_standard_stack`	Attach geometry, polarization, and reference-standard meaning to the forward model.
Forward simulation	`simulate_sample_from_reference`	Apply $H_{\mathrm{sample}}/H_{\mathrm{ref,std}}$ to the measured reference and return the modeled trace.
Noise injection	`noise_sigma_from_dynamic_range`, `add_white_gaussian_noise`	Generate synthetic noisy observations for studies and documentation examples.
Fit-parameter path mapping	`_parse_path`, `_get_by_path`, `_set_by_path`, `stack_path_from_user_path`, `apply_fit_values`	Map the optimizer vector into the nested resolved stack.
Objective and metrics	`objective_metric_value`, `mse`, `relative_l2`, `snr_db`	Score each simulated trace against the observed trace.
Optimization	`fit_sample_trace`	Run differential evolution plus bounded local refinement.

Calculation block	Primary functions	Role in the pipeline
Local covariance estimate	`_residual_vector`, `_step_from_bounds`, `_estimate_parameter_sigmas`, `_correlation_from_covariance`	Approximate local uncertainties and parameter correlations from a finite-difference Jacobian.

12 Literature Anchors Used for the Implementation

The implementation decisions in this refactor were cross-checked against the local PDF set in `C:\Users\cs7bp\Desktop\THz_application_sim - Copy`. The most relevant anchors were:

- `diss_hoffmann.pdf`: PCA emission is strongly linearly polarized, which supports the single measured scalar channel used here.
- `s10762-024-01011-x.pdf`: oblique-incidence THz analysis requires explicit `s/p` handling and careful reference treatment.
- `THz-TDS_Time-Trace_Analysis_for_the_Extraction_of_Material_and_Metamaterial_Parameters.pdf` and `1810.12567v4 (1).pdf`: time-trace fitting is a natural and physically meaningful objective for THz-TDS.
- `s10762-025-01052-w.pdf` and related THz-TDS transform notes: Fourier convention, phase handling, and time-origin bookkeeping matter for robust fitting.

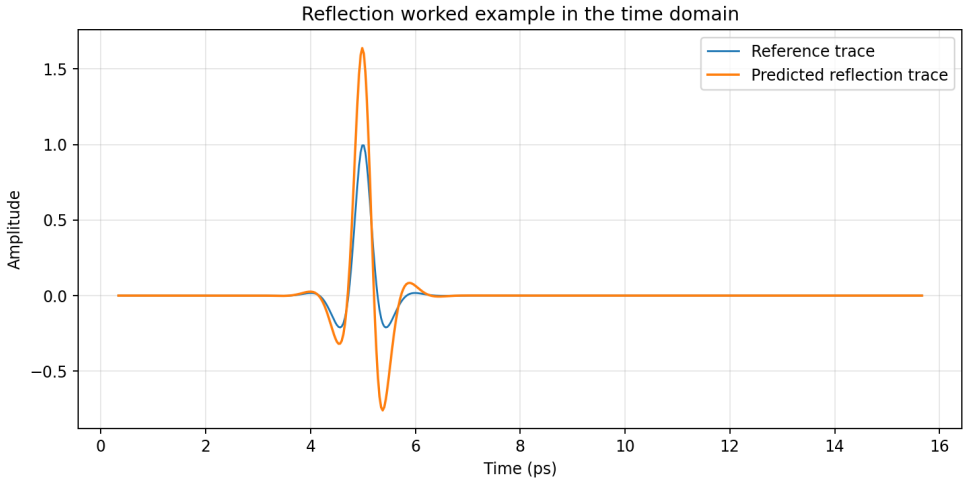

Figure 7: Reflection worked example in the time domain.